

CTF Writeup: Paragraphs

Challenge: Base64.docm + flag2.zip

Category: Forensics

Flag: SK-CERT{M5WORD_F0R3N51C5}

Assembly Order: 4-3-1-2

1. Overview

The challenge involves a Word macro-enabled document (Base64.docm) and a password-protected ZIP file (flag2.zip). The objective is to locate four hidden message parts within the document and assemble them in the order 4-3-1-2 to retrieve the ZIP password. The visible content is a distraction focused on Base64 encoding.

2. Initial Analysis

Modern Office files like .docm are ZIP archives. The investigation begins by extracting the file's internal structure:

```
unzip Base64.docm -d extracted/
```

Initial searches for obvious strings like "HIDDEN_MSG" or "FLAG" yielded no direct results, indicating more creative hiding methods.

3. Hidden Message Discovery

Part 1: Invisible Pixel Data (MSG_1 = 03c77a9b)

- **Location:** word/media/image39.png.
- **Discovery:** This image stood out due to its high resolution (1504x520) but suspiciously small file size (23KB), and it appeared blank when opened.
- **Technique:** Text was rendered in very light grey pixels (near RGB 255) against a white background, making it invisible to the naked eye.
- **Extraction:** Pixel-level analysis revealed a small cluster of non-white pixels. Thresholding and inverting the colors revealed the string HIDDEN_MSG_1_{03c77a9b}.

Part 2: Obfuscated XML String (MSG_2 = 47d0241a)

- **Location:** word/document.xml.
- **Discovery:** A Base64 string was buried within the complex XML markup of the document body.
- **Technique:** The string SEIEREVOX01TR18yX3s0N2QwMjQxYX0= was placed where internal relationship IDs or checksums are normally expected.
- **Extraction:** Decoding the string via Base64 yielded HIDDEN_MSG_2_{47d0241a}.

Part 3: Invisible Footer Text (MSG_3 = 5caf69d6)

- **Location:** word/footer1.xml.
- **Discovery:** The footer appeared empty in Word, but the XML revealed character-by-character hidden text.
- **Technique:** Two layers of hiding were used: white font color (FFFFFF) on a white background and an extremely small 2pt font size.
- **Extraction:** Direct inspection of the XML run elements revealed HIDDEN_MSG_3_{5caf69d6}.

Part 4: VBA Macro Binary (MSG_4 = 1ff1519f)

- **Location:** word/vbaProject.bin.
- **Discovery:** As a .docm file, it contains binary VBA macro code.
- **Technique:** A Base64 string literal was embedded within the binary project file, appearing as part of a legitimate automation macro.
- **Extraction:** Running the strings command on the binary file pulled out SEIEREVOX01TR180X3sxZmYxNTE5Zn0=, which decodes to HIDDEN_MSG_4_{1ff1519f}.

4. Password Assembly and Final Flag

Assembling the parts in the required order (4-3-1-2) provides the password for flag2.zip:

- **MSG_4:** 1ff1519f
- **MSG_3:** 5caf69d6
- **MSG_1:** 03c77a9b
- **MSG_2:** 47d0241a

ZIP Password: 1ff1519f5caf69d603c77a9b47d0241a

Unzipping the file reveals the final flag: **SK-CERT{M5W0RD_F0R3N51C5}**.

5. Key Takeaways

- **Extract Everything:** Treat Office files as ZIP archives and examine all internal XML and media files.
- **Pixel Auditing:** Blank-looking images with large resolutions and small file sizes are prime candidates for steganography.
- **XML & Binary Inspection:** Always check footers, headers, and VBA binaries for Base64 encoded strings or hardcoded literals.